Informe: Proyecto Palworld

04/06/2024

—

Daniel Alejadandro Rodríguez Herrera
1-DAM

Estructura de código

El CRUD del proyecto está basado en JDBC que opera en el recurso palworld.db. Dicho CRUD se realiza mediante Clases DAO que utilizan querys de SQLite3.

MAIN

```
C:.
├───java
│   └───es
│       └───ies
│           └───puerto
│               ├───config
│               ├───dto
│               ├───exception
│               ├───mapper
│               ├───modelo
│               │   ├───abstractas
│               │   ├───dao
│               │   └───entities
│               ├───repositories
│               └───services
└───resources
    └───WEB-INF
```

CRUD

Este es uno de los dao, la estructura es similar en el resto.

```
package es.ies.puerto.modelo.dao;


import es.ies.puerto.exception.PalworldException;
import es.ies.puerto.modelo.abstractas.DaoAbstract;
import es.ies.puerto.modelo.entities.Personaje;


import java.sql.ResultSet;
```

```
import java.sql.SQLException;
import java.sql.Statement;
import java.util.HashSet;
import java.util.Set;

public class PersonajeDAO extends DaoAbstract {
   public PersonajeDAO() throws PalworldException {
   }

   public Set<Personaje> findAllPersonaje() throws PalworldException
{
       String query;
       query  = "select p.personaje_id, p.usuario from Personaje as
p";
       return obtener(query);
   }

   public Personaje findPersonaje(Personaje personaje) throws
PalworldException {
       String query = "select p.personaje_id, p.usuario from
Personaje as p" +
               " where p.id='"+personaje.getPersonajeId()+"'";
       Set<Personaje> lista = obtener(query);
       if(lista.isEmpty()) {
           return null;
       }
       return lista.iterator().next();
   }

   public boolean updatePersonaje(Personaje personaje) throws
PalworldException {
```

```
        String query = "INSERT INTO Personaje as p (personaje_id,
usuario)" +
                " VALUES
('"+personaje.getPersonajeId()+"'"+personaje.getUsuario()+"')";
        Personaje findPersonaje = findPersonaje(personaje);
        if (findPersonaje!= null) {
            query = "update Personaje set
usuario='"+personaje.getUsuario()+"'" +
                    " where
personaje_id='"+personaje.getPersonajeId()+"'";
        }

        //Si existe actualiza
        //Si NO existe inserta
        actualizar(query);
        return false;
    }

    public void deletePersonaje(Personaje Personaje) throws
PalworldException {
        String query = "delete FROM Personaje as p" +
                " where
p.personaje_id='"+Personaje.getPersonajeId()+"'";
        actualizar(query);
    }

    private Set<Personaje> obtener(String query) throws
PalworldException {
        Set<Personaje> lista = new HashSet<>();
        Statement statement = null;
```

```
        ResultSet rs = null;
        try {
            statement = getConexion().createStatement();
            rs = statement.executeQuery(query);
            while (rs.next()) {
                String personajeId = rs.getString("personaje_id");
                String usuario = rs.getString("usuario");

                Personaje Personaje = new
Personaje(personajeId,usuario,null, null,null);
                lista.add(Personaje);
            }
        } catch (SQLException exception) {
            throw new PalworldException(exception.getMessage(),
exception);
        } finally {
            try {
                if (rs != null && !rs.isClosed()) {
                    rs.close();
                }
                if (statement != null && !statement.isClosed()) {
                    statement.close();
                }
                if (!getConexion().isClosed()) {
                    getConexion().close();
                }
            } catch (SQLException e) {
                throw new PalworldException(e.getMessage(), e);
            }
        }
```

```
        return lista;
    }
}
```

MAPPERS

Este es uno de los mappers, la estructura es similar en el resto.

```
public class MaterialMapper {
    1 usage
    public static MaterialDTO materialToMaterialDTO(Material material){
        if (material == null){
            return null;
        }
        MaterialDTO materialDTO = new MaterialDTO();
        materialDTO.setNombre(material.getNombre());
        materialDTO.setCantidad(material.getCantidad());
        materialDTO.setPersonajeId(material.getPersonajeId());

        return materialDTO;
    }

    1 usage
    public static Material materialDTOTOToMaterial(MaterialDTO materialDTO) {
        if (materialDTO == null) {
            return null;
        }
        Material material = new Material();
        material.setNombre(materialDTO.getNombre());
        material.setCantidad(materialDTO.getCantidad());
        material.setPersonajeId(materialDTO.getPersonajeId());

        return material;
```

SERVICE

Este es una de las clases repositories que utilizaran las clases services.

```
package es.ies.puerto.repositories;



import es.ies.puerto.modelo.entities.Pal;
```

```
import java.util.List;

public class PalRepository {
   private List<Pal> pals;

   public Pal findById(String id){
       Pal pal =new Pal(id);
       int indice = pals.indexOf(pal);
       if (indice>=0){
           return pals.get(indice);
       }
       return null;
   }
   public List<Pal> findAll(){return pals;}
   public boolean save(Pal pal){
       if (!pals.contains(pal)) {
           return pals.add(pal);
       }
       return false;
   }

   public boolean delete(String id){
       Pal pal= new Pal(id);
       return pals.remove(pal);
   }
}
```

Esta es una de las clases service del proyecto.

```
package es.ies.puerto.services;




import es.ies.puerto.modelo.entities.Pal;
import es.ies.puerto.repositories.PalRepository;

import javax.ws.rs.*;
import javax.ws.rs.core.Response;
import java.util.List;

@Path("/")
@Consumes("application/json")
@Produces("application/json")
public class PalService {
   private PalRepository palRepository;

   public void setPalRepository(PalRepository palRepository) {
       this.palRepository = palRepository;
   }

   public PalService() {
       this.palRepository = new PalRepository();
   }

   @GET
   @Path("/{id}")
   public Response findById(@PathParam("id")String  id){
       Pal pal=palRepository.findById(id);
```

```
        if (pal != null){
            return Response.ok(Response.accepted()).build();
        }else {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
    }

    @GET
    @Path("/")
    public Response findAll(){
        List<Pal> pals=palRepository.findAll();
        return Response.ok(pals).build();
    }

    @POST
    public Response save(Pal pal){
        boolean resultado=palRepository.save(pal);
        if (resultado){
            return Response.status(Response.Status.CREATED).build();
        }
        return Response.status(Response.Status.NOT_MODIFIED).build();
    }

    @DELETE
    @Path("/{id}")
    public Response delete(String id){
        boolean deleted=palRepository.delete(id);
        if (deleted) {
            return
Response.status(Response.Status.NO_CONTENT).build();
```

```
        } else {
            return Response.status(Response.Status.NOT_FOUND).build();
        }
    }
}
```

Test

CRUD

El test del mapper anterior. Todos los DAOTest mantienen la misma estructura.

```
package es.ies.puerto.modelo.dao;

import es.ies.puerto.modelo.entities.Personaje;
import org.junit.Test;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;

import java.util.Set;

public class PersonajeDAOTest {
    static PersonajeDAO daoPersonaje;
    final String idPersonaje="idPersonaje";
    final String usuario="usuario";

    final String nombrePersonajeUpdate="nombrePersonajeUpdate";
    Personaje personaje;
```

```
    @BeforeAll
    public static void beforeAll() {
        try {
            daoPersonaje = new PersonajeDAO();
        }catch (Exception exception) {
            Assertions.fail("Se ha producido un
error:"+exception.getMessage());
        }
    }

    @BeforeEach
    public void beforeEach() {
        try {
            Personaje personaje= new
Personaje(idPersonaje,usuario,null,null,null);
            daoPersonaje.updatePersonaje(personaje);
        }catch (Exception e) {
            Assertions.fail(e.getMessage());
        }

    }

    @Test
    public void findAllPersonajeTest() {
        try {
            Set<Personaje> lista = daoPersonaje.findAllPersonaje();
            Assertions.assertNotNull(lista);
            Assertions.assertTrue(lista.size()>0);
        }catch (Exception e) {
```

```
            Assertions.fail(e.getMessage());
        }
    }

    @Test
    public void findPersonajeTest() {
        try {
            Personaje PersonajeFind =
daoPersonaje.findPersonaje(personaje);
            Assertions.assertNotNull(PersonajeFind);
            Assertions.assertEquals(personaje, PersonajeFind);
        }catch (Exception e) {
            Assertions.fail(e.getMessage());
        }
    }

    @Test
    public void updatePersonajeTest() {
        try {
            Personaje PersonajeFind =
daoPersonaje.findPersonaje(personaje);
            Assertions.assertNotNull(PersonajeFind);
            PersonajeFind.setPersonajeId(nombrePersonajeUpdate);
            daoPersonaje.updatePersonaje(PersonajeFind);
            Personaje PersonajeFindUpdate =
daoPersonaje.findPersonaje(personaje);
            Assertions.assertNotNull(PersonajeFindUpdate);
            Assertions.assertEquals(PersonajeFind,
PersonajeFindUpdate);
        }catch (Exception e) {
```

```
            Assertions.fail(e.getMessage());
        }
    }



    @AfterEach
    public void afterEach()  {
        try {
            daoPersonaje.deletePersonaje(personaje);
        }catch (Exception e) {
            Assertions.fail(e.getMessage());
        }
    }
}
```

```
PersonajeDAOTest.java
70              Assertions.assertNotNull(PersonajeFindUpdate);
71              Assertions.assertEquals(PersonajeFind, PersonajeFindUpdate);
72          }catch (Exception e) {
73              Assertions.fail(e.getMessage());
74          }
75      }
76
77
78      @AfterEach
79      public void afterEach()  {
80          try {
81              daoPersonaje.deletePersonaje(Personaje);
82          }catch (Exception e) {
83              Assertions.fail(e.getMessage());
84          }

Run   PersonajeDAOTest
PersonajeDAOTest (e 640 ms      Tests passed: 3 of 3 tests – 640 ms
  findPersonajeTest 547 ms      C:\Users\anabe\.jdks\corretto-11.0.21\bin\java.exe ...
  updatePersonajeTe 64 ms       SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
  findAllPersonajeTe: 29 ms     SLF4J: Defaulting to no-operation (NOP) logger implementation
                                SLF4J: See http://www.slf4j.org/codes.html#StaticLoggerBinder for further details.
```

MAPPERS

El test del mapper anterior. Todos los mapperTest mantienen la misma estructura.

```
package es.ies.puerto.mapper;



import es.ies.puerto.dto.MaterialDTO;
import es.ies.puerto.modelo.entities.Material;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

public class MaterialMapperTest {
   Material material;
   Material materialMapper;
   MaterialDTO materialDTO;
   MaterialDTO materialDTOMapper;

   @BeforeEach
   public void beforeEach() {
       materialDTO = new MaterialDTO();;
       materialDTO.setNombre("NOMBRE TEST");
       materialDTO.setCantidad(40);
       materialDTO.setPersonajeId("ID TEST");



       material = new Material();
       material.setNombre("nombre test");
       material.setCantidad(55);
       material.setPersonajeId("id test");
```

```
    }
```

```
    @Test
    public void materialToMaterialDTOTest() {
        materialDTOMapper = MaterialMapper.materialToMaterialDTO(material);
        Assertions.assertEquals(material.getNombre(), materialDTOMapper.getNombre(),  message: "Resultado inesperado");
        Assertions.assertEquals(material.getCantidad(), materialDTOMapper.getCantidad(),  message: "Resultado inesperado");
        Assertions.assertEquals(material.getPersonajeId(), materialDTOMapper.getPersonajeId(),  message: "Resultado inesper

    }

    @Test
    public void materialDTOTOToMaterialTest() {
        materialMapper = MaterialMapper.materialDTOTOToMaterial(materialDTO);
        Assertions.assertEquals(materialDTO.getNombre(), materialMapper.getNombre(),  message: "Resultado inesperado");
        Assertions.assertEquals(materialDTO.getCantidad(), materialMapper.getCantidad(),  message: "Resultado inesperado");
        Assertions.assertEquals(materialDTO.getPersonajeId(), materialMapper.getPersonajeId(),  message: "Resultado inesper

    }
```

SERVICE

Este es el test del service anterior. Todos los serviceTest mantienen la misma estructura.

```
package es.ies.puerto.service;



import es.ies.puerto.modelo.entities.Pal;
import es.ies.puerto.repositories.PalRepository;
import es.ies.puerto.services.PalService;
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

import javax.ws.rs.core.Response;
import java.util.ArrayList;

import static org.mockito.ArgumentMatchers.any;
```

```
import static org.mockito.ArgumentMatchers.anyString;
import static org.mockito.Mockito.when;

public class PalServiceTest {
   PalService palService;
   @Mock
   PalRepository palRepositoryMock;

   @BeforeEach
   public void init() {
       MockitoAnnotations.openMocks(this);
       palService=new PalService();
       palService.setPalRepository(palRepositoryMock);
   }

   @Test
   void findByIdTest(){
       when(palRepositoryMock.findById(anyString())).thenReturn(new
Pal());
       Response respuesta=palService.findById("1");
       Assertions.assertNotNull(respuesta);
       Assertions.assertEquals(200,respuesta.getStatus());
   }

   @Test
   void findByIdNotFoundTest(){

when(palRepositoryMock.findById(anyString())).thenReturn(null);
       Response respuesta=palService.findById("1");
       Assertions.assertNotNull(respuesta);
```

```
        Assertions.assertEquals(404,respuesta.getStatus());
    }

    @Test
    void findAllTest(){
        when(palRepositoryMock.findAll()).thenReturn(new
ArrayList<>());
        Response respuesta = palService.findAll();
        Assertions.assertNotNull(respuesta);
        Assertions.assertEquals(200,respuesta.getStatus());
    }

    @Test
    public void saveTest(){
        when(palRepositoryMock.save(any(Pal.class))).thenReturn(true);
        Response respuesta = palService.save(new Pal());
        Assertions.assertNotNull(respuesta);
        Assertions.assertEquals(201,respuesta.getStatus());
    }

    @Test
    void saveDuplicateTest() {

when(palRepositoryMock.save(any(Pal.class))).thenReturn(false);
        Response respuesta = palService.save(new Pal());
        Assertions.assertNotNull(respuesta);
        Assertions.assertEquals(304,respuesta.getStatus());
    }
    @Test
    void deleteOneOkTest() {
```

```
        when(palRepositoryMock.delete(anyString())).thenReturn(true);
        Response respuesta = palService.delete("3");
        Assertions.assertNotNull(respuesta);
        Assertions.assertEquals(204,respuesta.getStatus());
    }

    @Test
    void deleteOneErrorTest() {
        when(palRepositoryMock.delete(anyString())).thenReturn(false);
        Response respuesta = palService.delete("1k");
        Assertions.assertNotNull(respuesta);
        Assertions.assertEquals(404,respuesta.getStatus());
    }
}
```

```
import ...

public class PalServiceTest {
    9 usages
    PalService palService;
    8 usages
    @Mock
    PalRepository palRepositoryMock;

    @BeforeEach
    public void init() {
        MockitoAnnotations.openMocks( testClass: this);
        palService=new PalService();
        palService.setPalRepository(palRepositoryMock);
    }
```

```
Run    PalServiceTest
PalServiceTes 2 sec 125 ms        Tests passed: 7 of 7 tests – 2 sec 125 ms
  saveTest() 2 sec 102 ms         C:\Users\anabe\.jdks\corretto-11.0.21\bin\java.exe ...
  deleteOneOkTestI 6 ms
  findByIdNotFound 4 ms           Process finished with exit code 0
  deleteOneErrorTe 4 ms
  findAllTest() 3 ms
```

Cobertura de código

Lamentablemente, ha surgido un conflicto con el Maven y el JDK, por lo que en el momento de la entrega del informe no se ha podido documentar la cobertura de código. Este error también me sucede al intentar ejecutar mvn en otros trabajos como las correcciones de los exámenes de clase. Sin embargo, todos los métodos están testeados y comprobados.